PySpark Structured Streaming

Socket Streaming and File Streaming

Detailed Beginner-Friendly Training Guide

Topics	Socket Stream, File Stream, Structured Streaming
Language	Python + PySpark
Audience	Beginners / Data Engineers / Python Developers
Examples	TCP socket server, PySpark socket reader, directory file reader

Table of Contents

1. 1. Introduction to Stream Processing
2. 2. PySpark Structured Streaming
3. 3. Common Streaming Architecture
4. 4. Socket Streaming
5. 5. Socket Streaming Architecture Flow
6. 6. Socket Streaming Example Using Python
7. 7. Running the Socket Example
8. 8. Understanding the Socket Code Step by Step
9. 9. File Streaming
10. 10. File Streaming Architecture Flow
11. 11. File Streaming Example Using Python
12. 12. Running the File Streaming Example
13. 13. Understanding the File Streaming Code
14. 14. Output Modes
15. 15. Checkpointing and Fault Tolerance
16. 16. Socket Streaming vs File Streaming
17. 17. Common Errors and Troubleshooting
18. 18. Practical Exercises
19. 19. Summary

1. Introduction to Stream Processing

Stream processing means processing data continuously as new data becomes available instead of waiting until all data has been collected. In a traditional batch process, an application may read a file once, process all records, and finish. In stream processing, the application keeps running and processes new records or files as they arrive.

Example scenario:

- A bank receives transaction events continuously.
- A website generates click events every second.
- An IoT device sends temperature readings continuously.
- A directory receives CSV files every few minutes.
- An application receives text messages through a TCP socket.

Batch vs Streaming

Batch Processing	Streaming	Example
Processes a fixed dataset	Processes continuously arriving data	CSV report vs live transactions
Usually starts and stops	Usually keeps running	Daily job vs real-time monitor
Higher latency is often acceptable	Low latency is usually important	Daily sales vs fraud alerts

2. PySpark Structured Streaming

PySpark Structured Streaming is Spark's high-level streaming API for processing continuously arriving data using DataFrame and SQL-style operations. The same concepts used with Spark DataFrames—select, filter, groupBy, join, aggregation, and transformations—can be applied to streaming DataFrames, subject to streaming-specific rules.

Important idea: streaming DataFrame

A streaming DataFrame represents an unbounded table. Unlike a normal DataFrame, it does not have a final end because new rows may continue to arrive.

```
# Batch DataFrame
df = spark.read.csv("data.csv", header=True)

# Streaming DataFrame
stream_df = spark.readStream.text("input/")
```

The second DataFrame is a logical representation of data that will arrive in the input directory over time.

3. Common Streaming Architecture

A typical streaming application follows this pattern:

```
SOURCE  →  STREAMING READ  →  TRANSFORMATIONS  →  STREAMING WRITE  →  SINK
            (Spark)               (filter/select)       (console/files/etc.)
```

- Source: where streaming data originates.
- Read: Spark continuously reads new data/events.
- Transformation: data is parsed, filtered, enriched, aggregated, or joined.
- Sink: processed data is written to a destination.
- Checkpoint: Spark stores progress information so the query can recover.

Examples of sources include TCP sockets, files, Kafka, and other supported streaming connectors. Examples of sinks include console output, files, Kafka, and supported databases/connectors.

4. Socket Streaming

Socket streaming is a simple way to demonstrate real-time streaming. A TCP server sends text data through a network socket, while a PySpark Structured Streaming application connects to that host and port and processes the incoming text.

It is especially useful for learning because the data flow is easy to visualize. However, the socket source is generally used for learning/testing rather than as a production-grade ingestion architecture.

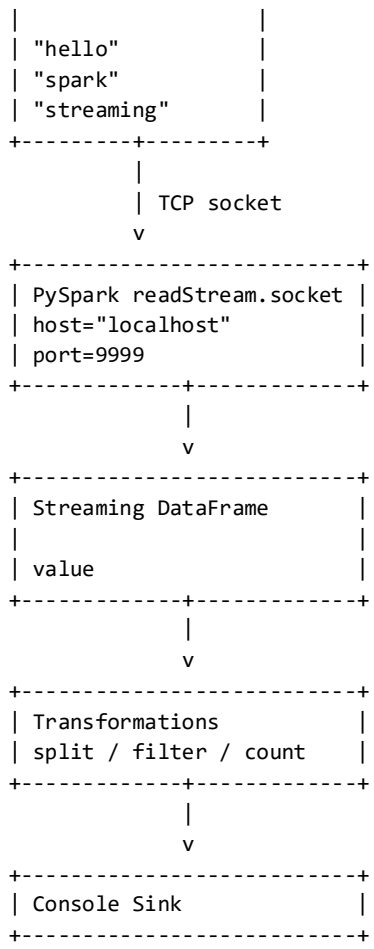
Key terms

- Host: the machine name or IP address where the TCP server is running.
- Port: the TCP port on which the server listens.
- Socket server: a small Python program that accepts a client connection and sends data.
- Socket source: Spark's streaming reader that receives text from the socket.
- Micro-batch: Spark can process newly available data in small batches during each trigger.

5. Socket Streaming Architecture Flow

```
SOCKET STREAMING

+--------------------+
| Python TCP Server |
```



Data flow example:

- Server sends: "hello spark"
- Spark receives the line as a row in the value column.
- Spark can split the text into words.
- A word-count transformation can calculate counts for words currently observed.
- The console sink displays the streaming result.

6. Socket Streaming Example Using Python

6.1 Prerequisites

```
pip install pyspark
```

You also need a compatible Java installation for your Spark version. For a local learning setup, make sure JAVA_HOME is correctly configured.

6.2 Create the TCP Server

Create a file named socket_server.py:

```
import socket
import time

HOST = "localhost"
```

```
PORT = 9999

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

server_socket.bind((HOST, PORT))
server_socket.listen(1)

print(f"Listening on {HOST}:{PORT}...")
conn, addr = server_socket.accept()

print("Client connected:", addr)

messages = [
    "hello spark",
    "pyspark streaming",
    "hello data engineering",
    "spark streaming",
    "python pyspark",
]

for message in messages:
    conn.sendall((message + "\n").encode("utf-8"))
    print("Sent:", message)
    time.sleep(2)

conn.close()
server_socket.close()
```

6.3 Create the PySpark Streaming Application

Create a file named socket_stream.py:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split

spark = (
    SparkSession.builder
    .appName("SocketStreamingExample")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

lines = (
    spark.readStream
    .format("socket")
    .option("host", "localhost")
    .option("port", 9999)
    .load()
)

words = lines.select(
    explode(split(lines.value, " ")).alias("word")
)

word_counts = (
    words.groupBy("word")
    .count()
)
```

```
query = (
    word_counts.writeStream
    .outputMode("complete")
    .format("console")
    .option("truncate", "false")
    .trigger(processingTime="2 seconds")
    .start()
)

query.awaitTermination()
```

6.4 What this application does

20. Creates a SparkSession.
21. Connects to localhost port 9999.
22. Reads incoming socket lines as a streaming DataFrame.
23. Splits each line into words.
24. Explodes the array so each word becomes a row.
25. Groups words and counts occurrences.
26. Writes the current result to the console.
27. Keeps running until the query is stopped.

7. Running the Socket Example

Open two terminal windows.

Terminal 1 — Start the PySpark application

```
python socket_stream.py
```

The Spark application will wait for data from port 9999.

Terminal 2 — Start the socket server

```
python socket_server.py
```

The server sends one message every two seconds. The Spark console should begin displaying word-count updates.

Typical console output will contain Spark logging plus a result table similar to:

```
-------------------------------------------
Batch: 0
-------------------------------------------

-------------------------------------------
Batch: 1
-------------------------------------------
+----------------+-----+
|word            |count|
+----------------+-----+
|hello           |2    |
|spark           |1    |
|pyspark         |1    |
+----------------+-----+
```

The exact output depends on trigger timing and when messages arrive, so the counts and batch numbers can vary.

8. Understanding the Socket Code Step by Step

8.1 Creating SparkSession

```
spark = (
    SparkSession.builder
    .appName("SocketStreamingExample")
    .getOrCreate()
)
```

SparkSession is the entry point for DataFrame and Structured Streaming operations.

8.2 Reading from a socket

```
lines = (
    spark.readStream
    .format("socket")
    .option("host", "localhost")
    .option("port", 9999)
    .load()
)
```

- readStream tells Spark that the source is streaming.
- format('socket') selects the socket source.
- host identifies the server machine.
- port identifies the TCP listening port.
- load() creates the streaming DataFrame.

8.3 Understanding the socket DataFrame

```
lines.printSchema()
```

For the socket text source, the main column is typically named value and contains the received text as strings.

8.4 split()

```
split(lines.value, " ")
```

This converts a line such as 'hello spark' into an array similar to ['hello', 'spark'].

8.5 explode()

```
explode(split(lines.value, " "))
```

explode converts array elements into separate rows. This makes word-level aggregation easy.

8.6 groupBy and count

```
words.groupBy("word").count()
```

This calculates how many times each word appears in the streaming result.

8.7 writeStream

```
word_counts.writeStream
    .outputMode("complete")
    .format("console")
    .start()
```

writeStream starts the streaming query. outputMode('complete') writes the complete aggregation result to the console whenever a trigger runs. The exact output behavior depends on the query and sink.

9. File Streaming

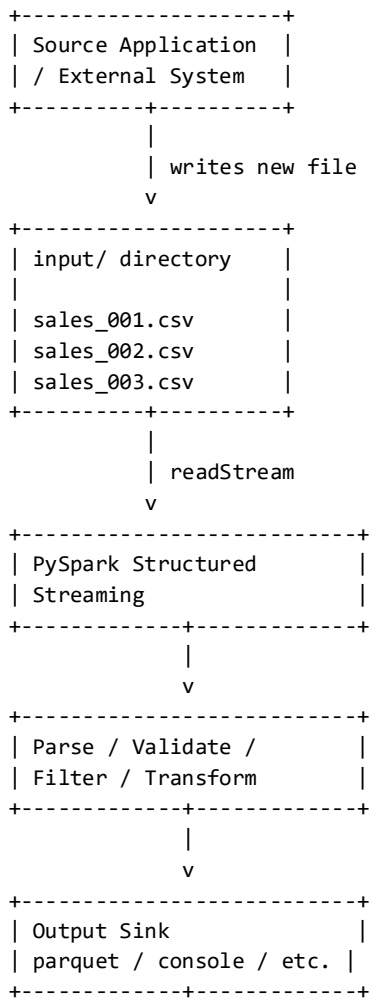
File streaming continuously monitors a directory and processes new files that arrive after the streaming query starts. A common learning pattern is to place CSV, JSON, or text files into an input directory while Spark is running.

For reliable file-stream processing, files should be completely written/closed before they are made visible to the monitored directory. In production, a controlled landing/staging process is commonly used.

Example scenario

- An application receives sales CSV files every five minutes.
- Files are placed in an input folder.
- Spark detects new files.
- Spark reads the new records.
- Spark filters and transforms the data.
- Spark writes processed results to an output location.

10. File Streaming Architecture Flow



```
              |
              v
+---------------------------+
| Checkpoint Location       |
+---------------------------+
```

Important concept: the streaming query does not repeatedly reread every old file as if it were a new file. Spark maintains progress information and processes newly discovered files according to the source's semantics and configuration.

11. File Streaming Example Using Python

11.1 Create the folder structure

```
streaming_demo/
    file_stream.py
    input/
    output/
    checkpoint/
```

11.2 Sample input CSV

```
order_id,customer,amount
101,John,2500
102,David,1800
103,Anita,3200
```

Save this as input/orders_001.csv after starting the streaming application, or place it there before startup depending on your test scenario. For a clear demonstration of newly arriving files, start Spark first and then copy new CSV files into the input directory.

11.3 PySpark file streaming code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = (
    SparkSession.builder
    .appName("FileStreamingExample")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

orders = (
    spark.readStream
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("input/")
)

high_value_orders = (
    orders
    .filter(col("amount") > 2000)
    .select("order_id", "customer", "amount")
)
```

```
query = (
    high_value_orders.writeStream
    .format("console")
    .outputMode("append")
    .option("truncate", "false")
    .option("checkpointLocation", "checkpoint/")
    .trigger(processingTime="5 seconds")
    .start()
)

query.awaitTermination()
```

11.4 What happens

28. Spark monitors the input directory.
29. When a new CSV file is detected, Spark reads its records.
30. The amount column is interpreted as a numeric value because inferSchema is enabled.
31. Rows with amount greater than 2000 are selected.
32. The selected rows are displayed in the console.
33. The checkpoint location stores progress information.

12. Running the File Streaming Example

34. Create input, output, and checkpoint directories.
35. Start the Python streaming program.
36. Wait until Spark is running.
37. Copy a new CSV file into the input directory.
38. Wait for the next trigger.
39. Observe the newly processed records in the console.
40. Add another new file and observe the next micro-batch.

Command

```
python file_stream.py
```

Example second input file

```
order_id,customer,amount
104,Meera,4500
105,Ravi,1200
106,Peter,2800
```

The filter keeps orders 104 and 106 because their amount is greater than 2000.

13. Understanding the File Streaming Code

13.1 readStream

```
orders = (
    spark.readStream
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("input/")
)
```

- readStream tells Spark this is a streaming read.
- format('csv') specifies the file format.
- header=true tells Spark the first row contains column names.
- inferSchema=true asks Spark to infer data types.
- load('input/') points to the monitored directory.

13.2 Filtering

```
high_value_orders = orders.filter(col("amount") > 2000)
```

The streaming DataFrame is filtered so only records satisfying the condition are passed downstream.

13.3 append output mode

```
.outputMode("append")
```

Append mode outputs only rows that are newly added to the result and are appropriate for many non-aggregated streaming transformations.

13.4 checkpointLocation

```
.option("checkpointLocation", "checkpoint/")
```

A checkpoint stores streaming query progress and related state. A stable checkpoint location is important for recovery and for maintaining progress across restarts. Do not casually reuse the same checkpoint directory for unrelated streaming queries.

14. Output Modes

Mode	Meaning	Typical Use	Important Note
append	Outputs newly added result rows	Non-aggregated transformations	Common for file/event flows
complete	Outputs the entire result table	Aggregations such as word count	Can produce more output
update	Outputs rows whose results changed	Some aggregations	Sink must support the mode

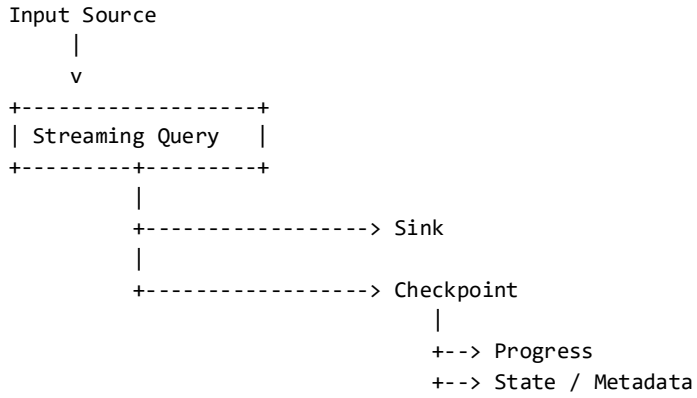
Not every output mode is valid for every query and sink. Aggregations, watermarks, joins, and the selected sink affect which modes can be used.

15. Checkpointing and Fault Tolerance

Checkpointing is one of the most important Structured Streaming concepts. Spark writes metadata about query progress and, where applicable, state information to the checkpoint directory.

- Allows a query to recover after certain failures.
- Helps Spark remember which input data has already been processed.
- Stores state for stateful operations when required.
- Should be stored in reliable persistent storage for production deployments.
- Should normally be dedicated to a specific streaming query.

Checkpoint architecture



16. Socket Streaming vs File Streaming

Feature	Socket Streaming	File Streaming	Typical Learning Use
Input	TCP socket	Directory of files	Understanding streaming sources
Data shape	Usually text lines	CSV/JSON/text/etc.	Text vs structured records
Setup	Requires a socket server	Requires input directory	Both are easy locally
Production use	Usually not recommended as a general production source	Common for file landing pipelines	Learning/demo
Main concept	Continuous event arrival	New-file detection	Streaming fundamentals

17. Common Errors and Troubleshooting

17.1 Connection refused

Symptom: Spark cannot connect to localhost:9999.

Cause: the socket server is not running, wrong port, or wrong host.

- Start the socket server before or around the time the Spark client connects.
- Confirm both applications use the same port.
- Check whether another process is using the port.

17.2 Address already in use

Symptom: Python reports that port 9999 is already in use.

- Stop the previous socket server.
- Use a different port such as 10000 and update the Spark code.
- On Windows, inspect the process using the port if necessary.

17.3 Java / JAVA_HOME errors

PySpark depends on the Java runtime used by Spark. Verify JAVA_HOME and the installed Java version are compatible with your Spark/PySpark setup.

17.4 Streaming aggregation output-mode errors

Some streaming aggregations cannot use append mode unless the query has the required conditions, such as watermarking and a supported aggregation pattern. For the simple socket word-count example, complete mode is used to make the changing aggregation visible.

17.5 Files are not being detected

- Confirm the path is correct.
- Confirm the new file is actually being copied into the monitored directory.
- Use a new filename for each test file.
- Make sure the file is fully written before Spark sees it.
- Check the Spark logs and query status.

17.6 Schema problems

- CSV files should have consistent structure when using a fixed streaming schema.
- For production pipelines, explicitly defining a schema is generally safer than relying on inference.
- Check column names, data types, delimiters, and malformed records.

18. Practical Exercises

Exercise 1 — Socket Word Count

- Modify the socket server to send 20 messages.
- Convert all incoming text to lowercase.
- Count words.
- Display the word counts every five seconds.

Exercise 2 — File Filtering

- Read streaming CSV order files.
- Filter amount > 5000.
- Select order_id, customer, and amount.
- Write the result to the console.

Exercise 3 — File Aggregation

- Group orders by customer.
- Calculate total amount.
- Experiment with output modes.
- Add a checkpoint directory.

Exercise 4 — Production-Oriented Extension

- Replace the console sink with a persistent output format.
- Define an explicit schema.
- Use a reliable checkpoint location.
- Add validation for malformed records.
- Consider how duplicate data and late-arriving data should be handled.

19. Summary

PySpark Structured Streaming provides a DataFrame-based programming model for continuously arriving data. Socket streaming is a simple educational source that demonstrates event-like data arriving over TCP. File streaming demonstrates a common pattern in which Spark processes new files arriving in a directory.

- Use readStream to create a streaming input.
- Use DataFrame transformations to process streaming data.
- Use writeStream to start a streaming query.
- Use output modes according to the query's semantics.
- Use checkpoints for recovery and progress tracking.
- For production systems, choose durable and supported ingestion technologies appropriate to the workload.
- Always test schema, failure recovery, duplicates, late data, and operational behavior.

Quick Reference

```
# Socket source
spark.readStream.format("socket")     .option("host", "localhost")     .option("port", 9999)
.load()

# File source
spark.readStream.format("csv")     .option("header", "true")     .schema(my_schema)
.load("input/")

# Start a query
df.writeStream     .format("console")     .outputMode("append")     .option("checkpointLocation",
"checkpoint/")     .start()

# Keep the application alive
query.awaitTermination()
```

End of Document